

02477 – Bayesian Machine Learning: Lecture 12

Michael Riis Andersen

Technical University of Denmark,
DTU Compute, Department of Applied Math and Computer Science

Variational inference in modern machine learning

- *Variational inference* (VI) has been the topic for the last two weeks
- Variational inference has many applications in modern machine learning
 1. Variational autoencoders (VAEs)
 2. Diffusion models
 3. Bayesian neural networks
 4. ...
- Common challenges
 1. Highly non-linear models
 2. Large-scale models
 3. Huge datasets
- Last bits of VI theory today: *Black-box variational inference* (BBVI)

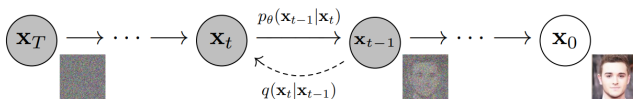


Figure 2: The directed graphical model considered in this work.

From Jonathan Ho et al (2020)

Outline

1 Variational inference and divergences

- Properties of the KL-divergence
- KL-divergences in other contexts (detour)

2 Black-box variational inference

- The evidence lower bound and entropy
- Dealing with expectations
- Stochastic gradients
- A few words on stochastic optimization
- Mini-batching for large scale inference

Variational inference and divergences

Variational inference and divergences: Properties of the KL-divergence

Measuring distance between probability distributions

- How do we measure "distance" between probability distributions $\mathbb{D}[q, p]$?
- The choice of "distance" affects the properties of the approximation
- The *Kullback-Leibler* divergence (for continuous R.V.) is defined as

$$\text{KL}[q||p] = \int q(z) \log \left[\frac{q(z)}{p(z)} \right] dz$$

■ Properties

1. Identity of indiscernibles

$$\text{KL}[q||p] = 0 \quad \Longleftrightarrow \quad p = q \quad (\text{a.e})$$

2. Non-negativity

$$\text{KL}[q||p] \geq 0$$

3. Asymmetric

$$\text{KL}[q||p] \neq \text{KL}[p||q]$$

Properties of the KL divergence

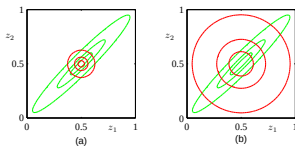
- The *variational approximation* q for target distribution $p \approx q$ is defined as

$$q_* = \arg \min_{q \in \mathcal{Q}} \text{KL} [q||p], \quad \text{KL} [q||p] = \int q(\mathbf{z}) \log \left[\frac{q(\mathbf{z})}{p(\mathbf{z})} \right] d\mathbf{z}$$

- Approximating a general Gaussian $p(\mathbf{w}) = \mathcal{N}(\mathbf{w}|\boldsymbol{\mu}, \boldsymbol{\Sigma})$ with a mean-field Gaussian

$$q(\mathbf{w}) = \mathcal{N}(w_1|m_1, v_1)\mathcal{N}(w_2|m_2, v_2)$$

- Which approximation q (in red) has the smallest KL $[q||p]$ divergence? (a) or (b)?



The approximation q in red
The target p distribution in green

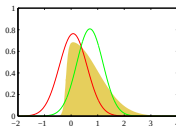
Common approximation properties

The *variational approximation* q for target distribution $p \approx q$ is defined as $q_* = \arg \min_{q \in \mathcal{Q}} \mathbb{D}[q||p]$

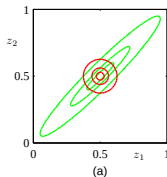
$$\text{KL}[q||p] = \int q(z) \log \left[\frac{q(z)}{p(z)} \right] dz$$

$$\text{KL}[p||q] = \int p(z) \log \left[\frac{p(z)}{q(z)} \right] dz$$

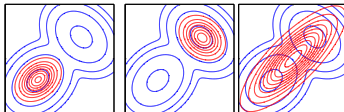
1. The variational approximation (green) often covers to posterior mass of target (yellow) better than the Laplace approximation (red)



2. For the combination of *factorized variational families* and $\text{KL}[q||p]$, the variational approximation (red) often *underestimates* the variance of the target distribution (green)



3. Approximations (red) based on $\text{KL}[q||p]$ are often said to be *mode-seeking*, whereas $\text{KL}[p||q]$ are often said to be *mass-covering*



The α -divergence

- The Kullback-Leibler divergence is not the only possible choice for the divergence

$$\text{KL}[q||p] = \int q(z) \log \frac{q(z)}{p(z)} dz$$

- The α -divergence is defined as

$$\mathbb{D}_\alpha[p||q] = \frac{1}{\alpha(1-\alpha)} \int \alpha p(z) + (1-\alpha)q(z) - p(z)^\alpha q(z)^{1-\alpha} dz$$

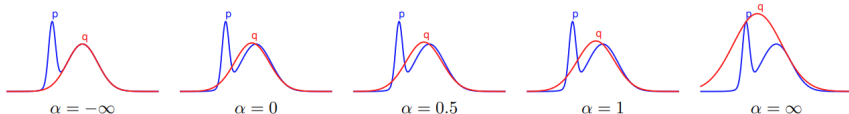
- Some interesting *special cases*

$$\lim_{\alpha \rightarrow 0} \mathbb{D}_\alpha[p||q] = \text{KL}[q||p]$$

$$\lim_{\alpha \rightarrow 1} \mathbb{D}_\alpha[p||q] = \text{KL}[p||q]$$

$$\mathbb{D}_{\frac{1}{2}}[p||q] = 2 \int \left(\sqrt{p(z)} - \sqrt{q(z)} \right) dz \quad (\text{Hellinger distance})$$

- α determines the properties of the resulting approximation $q^* = \arg \min_q \mathbb{D}_\alpha[p||q]$



Variational inference and divergences: KL-divergences in other contexts (detour)

De-tour: KL-divergences and maximum likelihood

- Suppose we have a data set $\mathcal{D} = \{\mathbf{x}_i\}_{i=1}^N$, where $\mathbf{x}_i \sim p_D$. Assume we want to fit some parametric distribution $p(\mathbf{x}|\theta)$ with parameters θ to the data $\mathcal{D}$.

$$\begin{aligned}\text{KL}[p_D(\mathbf{x})||p(\mathbf{x}|\theta)] &= \mathbb{E}_{p_D(\mathbf{x})} \left[\log \frac{p_D(\mathbf{x})}{p(\mathbf{x}|\theta)} \right] \\ &= \mathbb{E}_{p_D(\mathbf{x})} [\log p_D(\mathbf{x})] + \mathbb{E}_{p_D(\mathbf{x})} \left[\log \frac{1}{p(\mathbf{x}|\theta)} \right] \\ &= \mathbb{E}_{p_D(\mathbf{x})} \left[\log \frac{1}{p(\mathbf{x}|\theta)} \right] + K \\ &= -\mathbb{E}_{p_D(\mathbf{x})} [\log p(\mathbf{x}|\theta)] + K \\ &= -\frac{1}{N} \sum_{n=1}^N \log p(\mathbf{x}_n|\theta) + K\end{aligned}$$

- Therefore,

$$\hat{\theta}^* = \arg \min_{\theta} \text{KL}[p_D(\mathbf{x})||p(\mathbf{x}|\theta)] \approx \arg \max_{\theta} \sum_{n=1}^N \log p(\mathbf{x}_n|\theta) \equiv \hat{\theta}_{\text{MLE}}$$

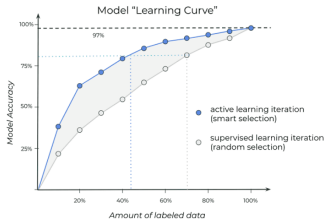
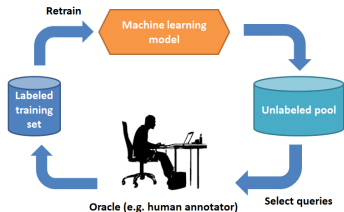
- Asymptotically ($N \rightarrow \infty$), *maximum likelihood learning* is equivalent to minimizing the *KL divergence between the true data distribution $p_D(\mathbf{x})$ and your model $p(\mathbf{x}|\theta)$* .

De-tour: Information gain

- Consider some Bayesian model with prior $p(\mathbf{w})$ and posterior $p(\mathbf{w}|\mathcal{D})$
- The KL divergence between the posterior and the prior is also sometimes called *information gain*

$$\text{KL}[p(\mathbf{w}|\mathcal{D})||p(\mathbf{w})]$$

- ... and quantifies how much *information* we gain by moving from the prior to the posterior
- Often used in *active learning* and *optimal experiment design* to choose the most informative subset of data for more data-efficient learning.



Li et al, 2018: Reversed Active Learning based Atrous DenseNet for Pathological Image Classification

<https://www.kdnuggets.com/2018/10/introduction-active-learning.html>

Black-box variational inference

Free-form and fixed-form variational inference

- In *free-form* VI, we use a *factorized approximation* for approximating the target distribution $p \equiv p(\mathbf{w}|\mathbf{y})$

$$q(\mathbf{w}) = \prod_{j=1}^J q(\mathbf{w}_j), \quad \text{where } \mathbf{w} = [\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_J]$$

- In *fixed-form* VI we choose a specific family distributions, e.g.

$$q(\mathbf{w}) = \mathcal{N}(\mathbf{w}|\mathbf{m}, \mathbf{V})$$

- ... and then evaluate and optimize the ELBO wrt. $q(\mathbf{w})$

$$\mathcal{L}[q] = \mathbb{E}_q[\log p(\mathbf{y}, \mathbf{w})] - \mathbb{E}_q[\log q(\mathbf{w})]$$

- | | |
|--|--|
| + Optimal functional form given assumptions (CAVI) | - Requires model-specific derivations |
| + Fast optimization | - Required integrals may be intractable |
| | - Optimal forms may not be "known" distributions |

Black-box variational inference

- How can we avoid the need for model-specific derivations? How can we avoid the restrictions on the choice of models we can approximate?
- Again, we use *fixed-form* families for continuous parameters, e.g. *mean-field* or *full-rank* Gaussians

$$q_{\text{MF}}(\mathbf{w}) = \prod_{i=1}^D \mathcal{N}(w_i | m_i, v_i) \qquad q_{\text{FR}}(\mathbf{w}) = \mathcal{N}(\mathbf{w} | \mathbf{m}, \mathbf{V})$$

- Optimize ELBO wrt. *variational parameters* $\lambda = \{\mathbf{m}, \mathbf{V}\}$

$$\lambda^* = \arg \max_{\lambda} \mathcal{L}[q_{\lambda}] = \arg \max_{\lambda} \{\mathbb{E}_{q_{\lambda}}[\log p(\mathbf{y}, \mathbf{w})] - \mathbb{E}_{q_{\lambda}}[\log q_{\lambda}(\mathbf{w})]\}$$

- We want to find an "automatic" method for

1. evaluating $\mathcal{L}[q_{\lambda}]$
2. computing gradients of $\mathcal{L}[q_{\lambda}]$

- ... allowing us to easily prototype, implement and test different models

Black-box variational inference: The evidence lower bound and entropy

The evidence lower bound and entropy

- The evidence lower bound $\mathcal{L}[q]$ is defined as

$$\mathcal{L}[q] = \mathbb{E}_q[\log p(\mathbf{y}, \mathbf{w})] - \mathbb{E}_q[\log q(\mathbf{w})] = \mathbb{E}_q[\log p(\mathbf{y}, \mathbf{w})] + \mathcal{H}[q]$$

- Second term is called the (differential) *entropy* of q , i.e. $\mathcal{H}[q] \equiv -\mathbb{E}_q[\log q(\mathbf{w})]$
- For "named" distributions, like Gaussians, the entropy is often easy to calculate or can be looked up.
- If $q(w_i) = \mathcal{N}(w_i | m_i, v_i)$, then

$$\mathcal{H}[q(w_i)] \equiv -\mathbb{E}_{q(w_i)}[\log q(w_i)] = \frac{1}{2} \log(2\pi e v_i)$$

- The entropy term and its gradient can often be computed analytically
- Therefore, we will focus on dealing with the expected log joint term: $\mathbb{E}_q[\log p(\mathbf{y}, \mathbf{w})]$

Quiz

Quiz time!

Spend 4 minutes answering the following questions in the quiz

DTU Learn - Quiz - Lecture12: Variational families

Black-box variational inference: Dealing with expectations

How to deal with the expected log joint

- We can approximate the first term using Monte Carlo samples:

$$\mathbb{E}_q [\log p(\mathbf{y}, \mathbf{w})] \approx \frac{1}{S} \sum_{s=1}^S \log p(\mathbf{y}, \mathbf{w}^{(s)}),$$

where

$$\mathbf{w}^{(s)} \sim q_{\lambda}(\mathbf{w}) \quad \text{for } s = 1, \dots, S$$

- Example: if $q_{\lambda}(\mathbf{w}) = \mathcal{N}(\mathbf{w}|\mathbf{m}, \mathbf{V})$, we sample $\mathbf{w}^{(s)} \sim \mathcal{N}(\mathbf{w}|\mathbf{m}, \mathbf{V})$ and plug them into the sum above
- Easy to implement and fits the bill
 1. All we need is to be able to evaluate to the log joint $p(\mathbf{y}, \mathbf{w})$
 2. Not restricted by existence of closed-form analytical results for expectations
 3. No model-specific derivations
- How about the gradients?

$$\nabla_{\lambda} \mathbb{E}_{q_{\lambda}} [\log p(\mathbf{y}, \mathbf{w})]$$

Black-box variational inference: Stochastic gradients

How to compute the gradients?

- First, we can use *Leibniz'* rule to change order of the gradient and the integral

$$\begin{aligned}\nabla_{\lambda} \mathbb{E}_{q_{\lambda}}[\log p(\mathbf{y}, \mathbf{w})] &\equiv \nabla_{\lambda} \int q_{\lambda}(\mathbf{w}) \log p(\mathbf{y}, \mathbf{w}) d\mathbf{w} \\ &= \int \nabla_{\lambda} q_{\lambda}(\mathbf{w}) \log p(\mathbf{y}, \mathbf{w}) d\mathbf{w}\end{aligned}$$

- We can't estimate the gradients directly using MC sampling. Why not?
- In order to estimate a quantity using MC, we need to be able to express it as an expectation. For example,

$$\mathbb{E}_q[\log p(\mathbf{y}, \mathbf{w})] = \int q(\mathbf{w}) \log p(\mathbf{y}, \mathbf{w}) d\mathbf{w} \approx \frac{1}{S} \sum_{s=1}^S \log p(\mathbf{y}, \mathbf{w}^{(s)}),$$

- To generate samples, we need a distribution, but $\nabla_{\lambda} q_{\lambda}(\mathbf{w})$ is not a distribution

The score function estimator

- *Log-derivative trick*: Using the chain-rule on the $\log q_{\lambda}(\mathbf{w})$ yields

$$\nabla_{\lambda} \log q_{\lambda}(\mathbf{w}) = \frac{1}{q_{\lambda}(\mathbf{w})} \nabla_{\lambda} q_{\lambda}(\mathbf{w})$$

- We can use this to re-write the gradient

$$\begin{aligned} \nabla_{\lambda} \mathbb{E}_{q_{\lambda}}[\log p(\mathbf{y}, \mathbf{w})] &\equiv \int \nabla_{\lambda} q_{\lambda}(\mathbf{w}) \log p(\mathbf{y}, \mathbf{w}) d\mathbf{w} \\ &= \int \frac{q_{\lambda}(\mathbf{w})}{q_{\lambda}(\mathbf{w})} \nabla_{\lambda} q_{\lambda}(\mathbf{w}) \log p(\mathbf{y}, \mathbf{w}) d\mathbf{w} \\ &= \int q_{\lambda}(\mathbf{w}) \frac{1}{q_{\lambda}(\mathbf{w})} \nabla_{\lambda} q_{\lambda}(\mathbf{w}) \log p(\mathbf{y}, \mathbf{w}) d\mathbf{w} \\ &= \int q_{\lambda}(\mathbf{w}) \nabla_{\lambda} \log q_{\lambda}(\mathbf{w}) \log p(\mathbf{y}, \mathbf{w}) d\mathbf{w} \quad (\text{log-derivative trick}) \\ &= \mathbb{E}_{q_{\lambda}}[\nabla_{\lambda} \log q_{\lambda}(\mathbf{w}) \log p(\mathbf{y}, \mathbf{w})] \quad (\text{Def. of expectation}) \\ &\approx \frac{1}{S} \sum_{s=1}^S \nabla_{\lambda} \log q_{\lambda}(\mathbf{w}^s) \log p(\mathbf{y}, \mathbf{w}^s) \quad \text{where} \quad \mathbf{w}^s \sim q_{\lambda} \end{aligned}$$

- This is called the *score function gradient estimator*

The score function gradient estimator

- The score function gradient estimator is defined as

$$\nabla_{\lambda} \mathbb{E}_{q_{\lambda}} [\log p(\mathbf{y}, \mathbf{w})] \approx \frac{1}{S} \sum_{s=1}^S \nabla_{\lambda} \log q_{\lambda}(\mathbf{w}^s) \log p(\mathbf{y}, \mathbf{w}^s) \quad \text{where} \quad \mathbf{w}^s \sim q_{\lambda}(\mathbf{w})$$

- $\nabla_{\lambda} \log q_{\lambda}$ can easily be calculated by hand or using automatic differentiation (e.g. Pytorch, Tensorflow, Autograd etc.)
- Only need to be able to evaluate $\log p(\mathbf{y}, \mathbf{w})$. No need for model-specific derivations
- General properties
 1. Very general and unbiased
 2. Applies to both continuous and discrete parameters
 3. Often high variance
- Big picture: approximating posterior distributions of general models
 1. We have chosen a variational family
 2. We aim to maximize the ELBO using gradient-based methods
 3. Estimate gradients using the score function estimator without the need for model-specific derivations
 4. Drawback: stochastic gradients

The BBVI algorithm

Putting everything together

Given a model and variational family (e.g. mean-field Gaussians)

$$p(\mathbf{y}, \mathbf{w}) = p(\mathbf{y}|\mathbf{w})p(\mathbf{w}), \quad q(\mathbf{w}) = \prod_{i=1}^D \mathcal{N}(w_i | m_i, v_i)$$

Black-box variational inference w. score function estimator

- Initialize variational parameters $\lambda_1 = \{\mathbf{m}_1, \log \mathbf{v}_1\}$
- For iteration $t = 1, \dots, T$ (or until convergence)
 1. Generate samples $\mathbf{w}^{(s)} \sim q(\mathbf{w}) = \mathcal{N}(\mathbf{w} | \mathbf{m}_t, \mathbf{v}_t)$ for $s = 1, \dots, S$
 2. Estimate ELBO

$$\hat{\mathcal{L}}[q] = \frac{1}{S} \sum_{s=1}^S \log p(\mathbf{y}, \mathbf{w}^{(s)}) + \mathcal{H}[q]$$

3. Estimate gradient

$$\nabla_{\lambda} \hat{\mathcal{L}}[q] \approx \frac{1}{S} \sum_{s=1}^S \nabla_{\lambda} \log q_{\lambda}(\mathbf{w}^s) \log p(\mathbf{y}, \mathbf{w}^s) + \nabla_{\lambda} \mathcal{H}[q]$$

4. Update variational parameters to get $\lambda_{t+1} = \{\mathbf{m}_{t+1}, \log \mathbf{v}_{t+1}\}$ using gradient estimate

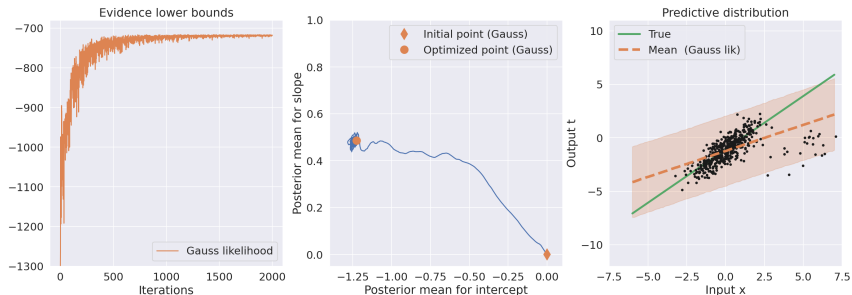
- Predictions & model checking: Evaluate training error, validation errors etc
- Go back and refine model if needed

Example: Robust regression I



$$p(\mathbf{y}, \mathbf{w}) = \prod_{n=1}^N \mathcal{N}(y_n | \mathbf{w}^T \mathbf{x}_n, \sigma^2) \mathcal{N}(\mathbf{w} | \mathbf{0}, \kappa^2 \mathbf{I})$$

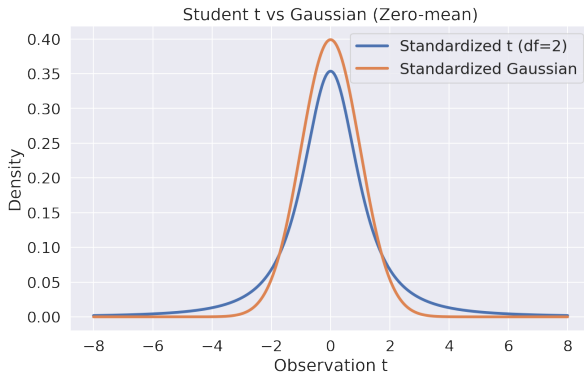
Example: Robust regression II



$$p(\mathbf{y}, \mathbf{w}) = \prod_{n=1}^N \mathcal{N}(y_n | \mathbf{w}^T \mathbf{x}_i, \sigma^2) \mathcal{N}(\mathbf{w} | \mathbf{0}, \kappa^2 \mathbf{I})$$

- Running BBVI with $S = 5$ samples for 2000 iterations

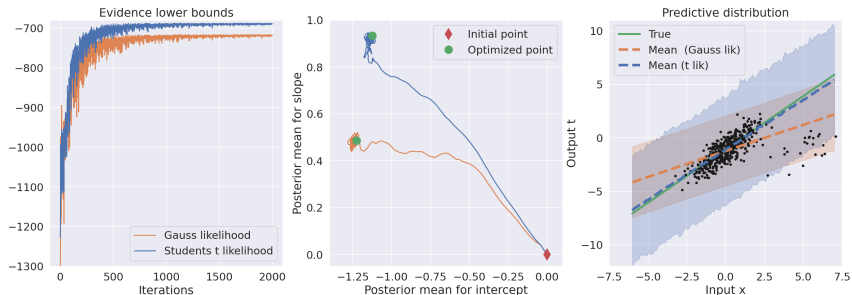
Example: Robust regression III



$$p_{\text{gauss}}(\mathbf{y}, \mathbf{w}) = \prod_{n=1}^N \mathcal{N}(y_n | \mathbf{w}^T \mathbf{x}_i, \sigma^2) \mathcal{N}(\mathbf{w} | \mathbf{0}, \kappa^2 \mathbf{I})$$

$$p_{\text{student-t}}(\mathbf{y}, \mathbf{w}) = \prod_{n=1}^N t_2(y_n | \mathbf{w}^T \mathbf{x}_i, \sigma^2) \mathcal{N}(\mathbf{w} | \mathbf{0}, \kappa^2 \mathbf{I})$$

Example: Robust regression IV

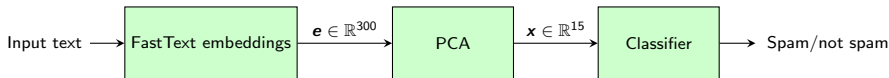


$$p_{\text{gauss}}(\mathbf{y}, \mathbf{w}) = \prod_{n=1}^N \mathcal{N}(y_n | \mathbf{w}^T \mathbf{x}_i, \sigma^2) \mathcal{N}(\mathbf{w} | \mathbf{0}, \kappa^2 \mathbf{I})$$
$$p_{\text{student-t}}(\mathbf{y}, \mathbf{w}) = \prod_{n=1}^N t_2(y_n | \mathbf{w}^T \mathbf{x}_i, \sigma^2) \mathcal{N}(\mathbf{w} | \mathbf{0}, \kappa^2 \mathbf{I})$$

- Running BBVI with $S = 5$ samples for 2000 iterations

Text classification: Spam detection

- In the exercise notebook, we will build a simple spam detector based on an SMS dataset
- Text examples
 1. "urgent! your mobile number has been awarded with a £2000 prize guaranteed. call 09061790121 from land line. claim 3030. valid 12hrs only 150ppm"
 2. "do we have any spare power supplies"
 3. "merry christmas to u too annie!"
- Transfer learning: Embed texts using FastText embedding (www.fasttext.cc)



- Which likelihood to use to gain robustness to outliers?

$$p(\mathbf{y}, \mathbf{w}) = \prod_{n=1}^N p(y_n | \mathbf{x}_n, \mathbf{w}) \mathcal{N}(\mathbf{w} | \mathbf{0}, \alpha^{-1} \mathbf{I})$$

Black-box variational inference: A few words on stochastic optimization

Simple example: Optimizing $J(\lambda)$

- Suppose $q(\mathbf{w}) = \mathcal{N}(\mathbf{w}|\lambda, \mathbf{I})$ and $\log p(\mathbf{y}, \mathbf{w}) = -\mathbf{w}^T \mathbf{w}$

$$J(\lambda) = \mathbb{E}_{q_\lambda} [\log p(\mathbf{y}, \mathbf{w})] = \mathbb{E}_{q_\lambda} [-\mathbf{w}^T \mathbf{w}]$$

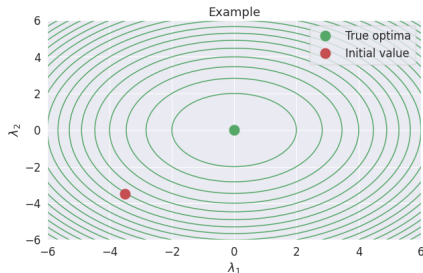
- We optimize J using gradient ascent with step-size $\rho = 0.1$

$$\lambda^{t+1} = \lambda^t + \rho \hat{g}(\lambda^t)$$

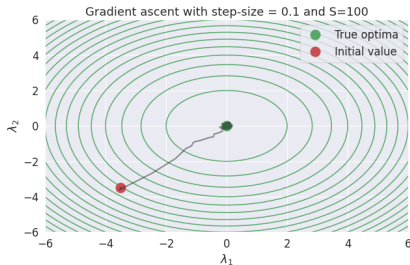
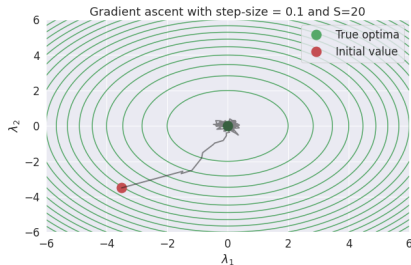
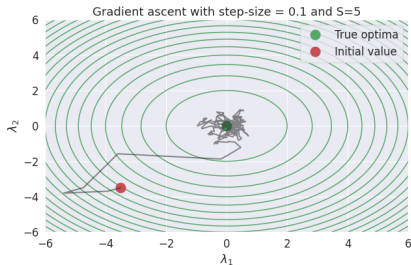
- Using the score function gradient estimator

$$\hat{g} = \frac{1}{S} \sum_{s=1}^S \nabla_{\lambda} \log q_{\lambda}(\mathbf{w}_s) \log p(\mathbf{y}, \mathbf{w}_s) = \frac{1}{S} \sum_{s=1}^S 2(\mathbf{w}_s - \lambda) \mathbf{w}_s^T \mathbf{w}_s$$

where $\mathbf{w}_s \sim q_{\lambda}(\mathbf{w})$



Simple example: Optimizing $J(\lambda)$ II



- Gradient ascent does not converge with constant step size
- The number of samples S trades off gradient variance vs computational cost

Stochastic gradient descent (SGD)

- The gradient ascent update with *constant step-size* η

$$\lambda^{t+1} = \lambda^t + \eta \hat{g}(\lambda^t)$$

- In *stochastic gradient ascent* the step-size is decreased in each iterations

$$\lambda^{t+1} = \lambda^t + \rho_t \eta \hat{g}(\lambda^t)$$

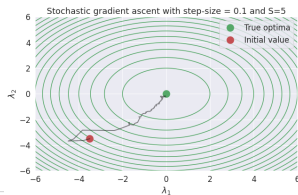
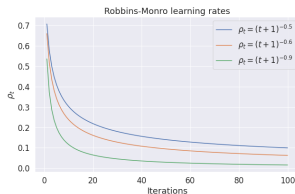
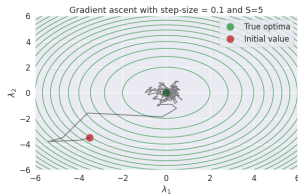
- The *Robbins-Monro conditions* guarantees convergence in probability if the gradient estimator is unbiased (and under regularity conditions)

$$\sum_{t=1}^{\infty} \rho_t = \infty \quad \text{and} \quad \sum_{t=1}^{\infty} \rho_t^2 < \infty$$

- Example: $\tau > 0$ and $\kappa \in (0.5, 1]$

$$\rho_t = (t + \tau)^{-\kappa}$$

Robbins & Monro (1951): A stochastic approximation method



A zoo of stochastic optimizers

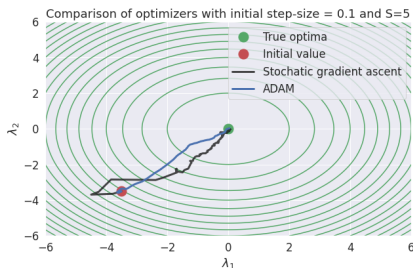
- Many recent methods for stochastic optimization: Adam, RMSProp, AdaGrad, AdaDelta etc
- Several key ideas for faster and more robust optimization
- **Momentum**: use gradient from previous iteration (rolling ball analogy)

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$
$$\lambda_{t+1} = \lambda_t + \eta m_t$$

- **Adaptive learning rates**: Adaptive learning rate based on magnitude of gradient

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$
$$\lambda_{t+1} = \lambda_t + \eta \frac{m_t}{\sqrt{v_t} + \epsilon}$$

- **Individual learning**: Each parameter is allowed to have its own adaptive learning rate
- The ADAM optimizer combines all these ideas (along with a bias-correction) and is often a good first choice



Quiz time

Quiz time!

DTU Learn - Quiz - Lecture 12 - BBVI quiz

The re-parametrization trick: gradient estimator with lower variance

- Gradient estimators with lower variance leads to faster optimization, because *higher variance requires smaller step-sizes*. Often we can do better than the score function gradient
- If $q(\mathbf{w}) = \mathcal{N}(\mathbf{w}|\mathbf{m}, \mathbf{V})$ is Gaussian and if $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, then $\mathbf{w} = \mathbf{m} + \mathbf{L}\epsilon \sim \mathcal{N}(\mathbf{w}|\mathbf{m}, \mathbf{V})$, where $\mathbf{L}$ is the Cholesky factorization (matrix square root) of $\mathbf{V} = \mathbf{L}\mathbf{L}^T$.
- We *re-parametrize* $\mathbf{w}$ as $\mathbf{w}_\epsilon = g(\boldsymbol{\lambda}, \epsilon) = \mathbf{m} + \mathbf{L}\epsilon$

$$\nabla_{\boldsymbol{\lambda}} \mathbb{E}_{q(\mathbf{w})}[\log p(\mathbf{y}, \mathbf{w})] = \nabla_{\boldsymbol{\lambda}} \mathbb{E}_{q(\epsilon)}[\log p(\mathbf{y}, \mathbf{w}_\epsilon)] \quad (\text{Re-parametrization})$$

$$= \nabla_{\boldsymbol{\lambda}} \int q(\epsilon) \log p(\mathbf{y}, \mathbf{w}_\epsilon) d\epsilon \quad (\text{Def. of expectation})$$

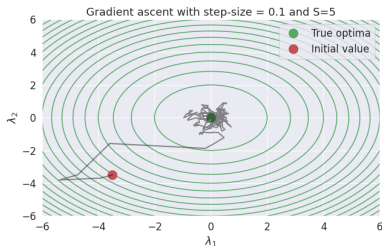
$$= \int q(\epsilon) \nabla_{\boldsymbol{\lambda}} \log p(\mathbf{y}, \mathbf{w}_\epsilon) d\epsilon \quad (\text{Leibniz' rule})$$

$$= \mathbb{E}_{q(\epsilon)}[\nabla_{\boldsymbol{\lambda}} \log p(\mathbf{y}, \mathbf{w}_\epsilon)] \quad (\text{Def. of expectation})$$

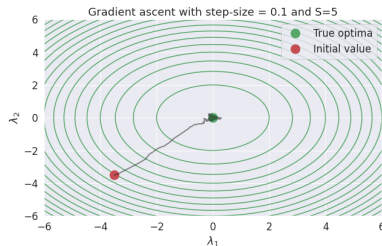
$$\approx \frac{1}{S} \sum_{s=1}^S \nabla_{\boldsymbol{\lambda}} \log p(\mathbf{y}, \mathbf{w}_{\epsilon_s}) \quad \text{where} \quad \epsilon_s \sim q(\epsilon)$$

- The *re-parametrized gradient estimator* requires the transformation g and $p(\mathbf{y}, \mathbf{w})$ to be differentiable, which rules out discrete variables.

Simple example revisited



Score function gradient estimator



Re-parametrized gradient estimator

- Comparison of the score function gradient estimator and the re-parametrized gradient estimator
- Using gradient ascent with fixed step-size for the purpose of illustration
- Score function is more general, but suffers from larger variance
- The re-parametrized gradient is less general and applies only to continuous variables, but has much lower variance

Reparametrized gradients

- Many variational families can be re-parametrized

Target	$p(z; \theta)$	Base $p(\epsilon)$	One-liner $g(\epsilon; \theta)$
Exponential	$\exp(-x); x > 0$	$\epsilon \sim [0; 1]$	$\ln(1/\epsilon)$
Cauchy	$\frac{1}{\pi(1+x^2)}$	$\epsilon \sim [0; 1]$	$\tan(\pi\epsilon)$
Laplace	$\mathcal{L}(0; 1) = \exp(- x)$	$\epsilon \sim [0; 1]$	$\ln(\frac{\epsilon_1}{\epsilon_2})$
Laplace	$\mathcal{L}(\mu; b)$	$\epsilon \sim [0; 1]$	$\mu - b \operatorname{sgn}(\epsilon) \ln(1 - 2 \epsilon)$
Std Gaussian	$\mathcal{N}(0; 1)$	$\epsilon \sim [0; 1]$	$\sqrt{\ln(\frac{1}{\epsilon_1})} \cos(2\pi\epsilon_2)$
Gaussian	$\mathcal{N}(\mu; RR^\top)$	$\epsilon \sim \mathcal{N}(0; 1)$	$\mu + R\epsilon$
Rademacher	$\operatorname{Rad}(\frac{1}{2})$	$\epsilon \sim \operatorname{Berm}(\frac{1}{2})$	$2\epsilon - 1$
Log-Normal	$\ln \mathcal{N}(\mu; \sigma^2)$	$\epsilon \sim \mathcal{N}(\mu; \sigma^2)$	$\exp(\epsilon)$
Inv Gamma	$i\mathcal{G}(k; \theta)$	$\epsilon \sim \mathcal{G}(k; \theta^{-1})$	$\frac{1}{\epsilon}$

Black-box variational inference: Mini-batching for large scale inference

Mini-batching for large scale inference

- Assuming a model of the form

$$p(\mathbf{y}, \mathbf{w}) = \prod_{n=1}^N p(y_n | \mathbf{w}) p(\mathbf{w})$$

- Let $\mathbf{w}^s \sim q_{\lambda}$ be samples from the variational approximation, then

$$\begin{aligned}\mathcal{L}[q] &\approx \frac{1}{S} \sum_{s=1}^S [\log p(\mathbf{y}, \mathbf{w}^s)] + \mathcal{H}[q] \\ &= \frac{1}{S} \sum_{s=1}^S \left[\sum_{n=1}^N \log p(y_n, \mathbf{w}^s) + \log p(\mathbf{w}^s) \right] + \mathcal{H}[q]\end{aligned}$$

- Evaluating the ELBO and its gradients becomes slow for large N
- **Mini-batching:** Let $\mathcal{M}$ be a random sample of the data points of size M , then the following is an unbiased estimator of the ELBO

$$\mathcal{L}[q] \approx \frac{1}{S} \sum_{s=1}^S \left[\frac{N}{M} \sum_{n \in \mathcal{M}} \log p(y_n, \mathbf{w}^s) + \log p(\mathbf{w}^s) \right] + \mathcal{H}[q]$$

- We can use the same ideas to speed up the calculations of the gradients, but the price is increased variance

Summary

Putting everything together

Given a model and variational family (e.g. mean-field Gaussians)

$$p(\mathbf{y}, \mathbf{w}) = p(\mathbf{y}|\mathbf{w})p(\mathbf{w}), \quad q(\mathbf{w}) = \prod_{i=1}^D \mathcal{N}(w_i | m_i, v_i)$$

Black-box variational inference w. the re-parametrized gradient

- Initialize variational parameters $\lambda_1 = \{\mathbf{m}_1, \log \mathbf{v}_1\}$
- For iteration $t = 1, \dots, T$ (or until convergence)
 1. Generate samples $\epsilon_s \sim \mathcal{N}(\epsilon | \mathbf{0}, \mathbf{I})$ for $s = 1, \dots, S$
 2. Estimate ELBO (use mini-batching if necessary)

$$\hat{\mathcal{L}}[q] = \frac{1}{S} \sum_{s=1}^S \log p(\mathbf{y}, \mathbf{w}_{\epsilon_s}) + \mathcal{H}[q], \quad \text{where } \mathbf{w}_{\epsilon_s} = \mathbf{m}_t + \mathbf{L}_t \epsilon_s$$

3. Estimate gradient (use mini-batching if necessary)

$$\nabla_{\lambda} \hat{\mathcal{L}}[q] \approx \frac{1}{S} \sum_{s=1}^S \nabla_{\lambda} \log p(\mathbf{y}, \mathbf{w}_{\epsilon_s}) + \nabla_{\lambda} \mathcal{H}[q]$$

4. Update variational parameters to get $\lambda_{t+1} = \{\mathbf{m}_{t+1}, \log \mathbf{v}_{t+1}\}$ using gradient estimate

- Predictions & model checking: Evaluate training error, validation errors etc
- Go back and refine model if needed